Stage 4 Presentation Script

When: Wednesday, October 28, 2026
Length: about 7 and a half minutes, including the clicking (limit is 5 to 8)
Speakers: Dylan (slides 1 to 5 and the live demo), Andrew (slides 6 to 9). Swap any section if you prefer; the words do not change.

Lines in quote blocks are said out loud, word for word. Lines marked **DO** are actions. Lines marked **LAUNCH** are terminal commands.

All commands run from the `~/Desktop/DB` folder:

```
cd ~/Desktop/DB
```

Part 1: Before the Talk

The night before (once)

The real data is already loaded on this laptop. To reload it from scratch (for example on another machine), run:

```
./demo.sh setup CPSC475/data/almanac.sqlite
```

It asks before dropping the database; answer `y`. Then check everything:

```
./demo.sh check
```

Every line should say `ok`. Write down the three values it prints:

Value	Write it here
Demo farm	
Harvest date to type	2025-08-12 (unless it says otherwise)
Market product to pick	Onions (unless it says otherwise)

The demo farm is a real Ohio farm with no records yet. The script picks it once and keeps using it.

Rehearse the demo at least once. After each rehearsal, clear the farm so it is empty again:

```
./demo.sh reset
```

15 minutes before the talk

1. **LAUNCH** Start the app:

`bash ./demo.sh`

This checks the database, starts the app at http://127.0.0.1:5000, and opens three browser tabs in order: **(1)** the home page, **(2)** the demo farm's overview, **(3)** crop search. Leave this terminal open for the whole talk; closing it stops the app.

1. Check the terminal output. If any line says `!!`, read it and do what it says (usually `./demo.sh reset`, then `./demo.sh` again).
2. **DO** In the browser, press `Ctrl` and `+` until the zoom is 125%, on each of the three tabs.
3. **DO** Click tab 1 (home page) so it is showing when you switch over.
4. **DO** Open `slides.pdf` in full screen on slide 1.
5. **DO** Silence your phone, close chat apps, and turn off notifications.

During the talk, switch between the slides and the browser with `Alt+Tab`.

Part 2: Slides (Dylan)

Slide 1: Almanac (0:00)

> "Good morning. We're Team Lesotho. I'm Dylan Montgomery, and this is Andrew Styles. Our project is Almanac, a farm record book that knows what your crops are worth."

Slide 2: Three kinds of data, one shared key (0:10)

> "Farm software keeps records of what you planted and harvested, but it has no prices, and it treats crops as free text. Auction reports have prices, but no connection to any farm. Plant databases have the taxonomy, but neither of the other two.
>
> Almanac ties all three together on one key: the taxon. Every grower event and every auction price points to a node in the same plant taxonomy, so a grower can ask what a harvest is worth in one query."

Slide 3: Collected, cleaned and loaded (0:40)

> "All of the reference data is real, and our scripts collect it from public sources: 33,667 auction prices from USDA reports for the Mount Hope Auction and from County Line Produce Auction, 520 sale days from 2019 to 2026. The taxonomy comes from GBIF, and the farms are 107 real Ohio farms from OpenStreetMap.
>
> If a row breaks a CHECK constraint, we don't drop it and we don't loosen the constraint. It goes into a quarantine table with the reason. One cabbage line had a low of 1 dollar 10, a high of 2 dollars 10, and an average of 19 dollars 50.
>
> Units are spelled 237 ways, so we standardize them to 35, and a price per peck is never averaged with a price per bushel. 85.8 percent of prices already match a product."

Slide 4: 12 tables, 15 foreign keys, 2 views (1:35)

> "The database has twelve tables. Eight hold reference data, and four hold grower data that users create, change and delete.
>
> We chose PostgreSQL for recursive queries and window functions. The SQL is written by hand with bound parameters, and data rules live in the schema, so they hold no matter which page writes. You'll see one in a minute."

Slide 5: The basic functions, in the app (2:00)

> "Let's see it running."

DO `Alt+Tab` to the browser. Tab 1, the home page, should be showing.

Part 3: Live Demo (Dylan)

Step 1: Real data (2:05) · requirement: real data

DO On the home page, point at the `market_lot` row, then the second table.

> "This is the live PostgreSQL database. Each number here is a COUNT on that table: 33,667 auction prices, 2,993 taxonomy nodes, 107 farms. The second table comes from a view, and shows how many prices from each source already match a product."

Step 2: Insert (2:30) · requirement: insert

DO Click tab 2, the demo farm.

> "This is a real Ohio farm from our data. It has no records yet, so I'll add some."

DO Click the **Crop catalog** tab. Under *Add a crop*, type `onion` and press **Search**.

> "One search box checks scientific names, common names and product titles. Results are ranked by how many auction lots each one has, so the onion we actually grow comes first."

DO Click **Add** on the *Allium cepa* row (it should be first).

DO Click the **Overview** tab, then **Add a plot**. Plot name `North field`, area `1200`. Click **Add plot**.

DO Click **Record an event**. Fill in:

Field	Value
Date	the harvest date you wrote down (2025-08-12)
Event	Harvest
Crop	onion
Plot	North field
Market product	the product you wrote down
Quantity	`40`
Unit	`lb`

Click **Record event**.

> "That's three inserts: a catalog entry, a plot, and a harvest. Each one is an INSERT with bound parameters."

Step 3: A database rule (3:40) · requirement: constraints

DO Click **Record an event** again. Leave the crop as onion, type `10` in Quantity, leave Unit blank, and click **Record event**. A red message appears: *An event with a quantity must also have a unit.*

> "That message comes from a CHECK constraint in the schema. The app catches the database error and explains it. Any page that writes an event is held to the same rule."

Step 4: Update (4:05) · requirement: update

DO Click the **Activity history** tab. On the harvest row, click **Edit**. Change Quantity to `55` and click **Save**.

> "That's an UPDATE. Every write is limited to this farm's ID, so one farm can't change another farm's records."

Step 5: Search and the multi-table join (4:25) · requirement: search

DO Click the **Harvest vs. market** tab.

> "This is our multi-table join. For each harvest, it joins the event log, farm, event kind, taxonomy, products, product aliases and auction lots, and returns what onion sold for within 30 days of our harvest, one row per unit. It matches on the taxon, not the name, so every onion lot counts, however the auction worded it."

DO Click tab 3, crop search. Type `onion`, press **Search**, and click **onion** on the *Allium cepa* row.

> "And this is the seasonal price curve: the floor, mean and ceiling price per peck in every month, across all eight seasons. The links above it switch units."

Step 6: Delete (5:20) · requirement: delete

DO Click tab 2. Click the **Overview** tab. In the Plots table, click **Delete** on *North field*, then **OK**.

> "Now I'll delete the plot."

DO Click the **Activity history** tab. Point at the harvest row: the Plot column is now blank.

> "The harvest is still here, just without a plot. The foreign key uses ON DELETE SET NULL, so a farm never loses its harvest history when it retires a field."

DO Click **Delete** on the harvest row, then **OK**.

> "And that's a delete on the event itself. Back to the slides."

DO `Alt+Tab` to the slides and go to slide 6.

Part 4: Slides (Andrew)

Slide 6: Four queries, four questions (5:55)

> "Here are the four search queries behind what you just saw. Crop search ranks matches by price evidence, the price curve groups lots by month, activity history filters a farm's events, and harvest versus market is the seven-table join."

Slide 7: Three things no farm app can do (6:10)

> "Next are our three advanced functions.
>
> First, a taxonomic price roll-up. Pick any node, like a plant family, and get prices for everything under it. Some species have subspecies or cultivar groups under them and others have none, so a fixed number of joins can't cover every branch. A recursive CTE walks any subtree in one query. On the order Asparagales it walks 305 nodes and finds 1,805 prices.
>
> Second, a crop decision engine. Given a planting window, it compares that window across every year, discounts each crop's price by how much it swings, and ranks crops with a window function.
>
> Third, data resolution. Our loaders fetch new reports automatically. The 1,445 labels they can't match go to a ladder of four rules. If a rule finds two matches, it marks the label ambiguous instead of guessing, and a person approves every match."

Slide 8: From here to the final demo (7:05)

> "The roll-up page is this week. The decision engine and the ladder are next week. The week of November 9 the site goes online and we start inviting growers to enter real records. Dylan is building the first two functions and the deployment. I'm building the ladder and its review screen."

Slide 9: Questions? (7:25)

> "That's Almanac. We're happy to take questions."

Part 5: Answers to Likely Questions

Question	Answer (either of us)
Why is the roll-up advanced?	"The user can start from any node, and the number of levels under it changes with the rank. A fixed number of joins can't express that, and it would break when we add varieties under species. The recursive CTE handles any depth in one statement."
Why not match auction labels by substring?	"Wild garlic isn't garlic. And when two products match equally well, picking whichever row comes back first gives a wrong answer. The ladder stops and reports it instead."
Where will grower data come from?	"Real users. The site goes public the week of November 9, and we'll email you when it's live."
How do you prevent SQL injection?	"Every value is a bound parameter. User input never becomes part of the SQL text."
Where does the data come from?	"Public sources, collected by our own scripts: the USDA Market News archive for Mount Hope, County Line's posted market reports, the GBIF plant taxonomy, and OpenStreetMap for farms. One command downloads and rebuilds all of it."
Why PostgreSQL and not MySQL?	"Recursive CTEs and window functions, which the first two advanced functions depend on, and strict CHECK enforcement."

If Something Goes Wrong

Problem	What to do
The browser says it can't connect	The app stopped. In the terminal, run `./demo.sh` again from `~/Desktop/DB`. It reopens the tabs.
Harvest vs. market is empty	The harvest date has no onion lots nearby. Click **Activity history**, **Edit** the harvest, and change the date to a month that has lots on the price curve.
The farm already has records when you start	A rehearsal was not cleared. Run `./demo.sh reset`, then refresh the page.
Running long	Skip Step 3 and the price curve. Insert, update, delete and the join are what's required.

After the Talk

In the terminal running the app, press `Ctrl+C` to stop it. Run `./demo.sh reset` if you want the demo farm empty again.